1594. Maximum Non Negative Product in a Matrix

You are given a m x n matrix grid. Initially, you are located at the top-left corner (0, 0), and in each step, you can only **move right or down** in the matrix.

Among all possible paths starting from the top-left corner (0, 0) and ending in the bottom-right corner (m - 1, n - 1), find the path with the **maximum non-negative product**. The product of a path is the product of all integers in the grid cells visited along the path.

Return the *maximum non-negative product* ***modulo*** $10^9 + 7$. *If the maximum product is* ***negative****, return* -1.

Notice that the modulo is performed after getting the maximum product.

Example 1:

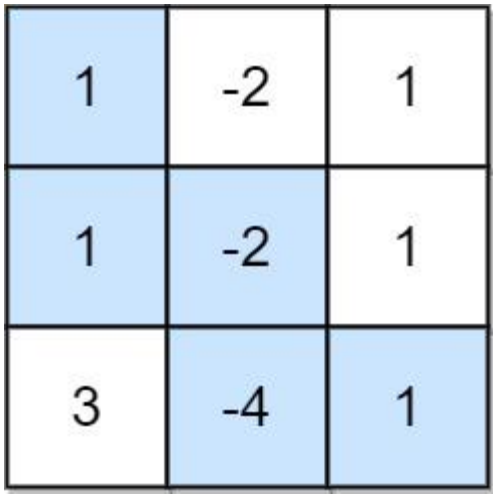
-1	-2	-3
-2	-3	-3
-3	-3	-2

Input: grid = [[-1,-2,-3],[-2,-3,-3],[-3,-3,-2]]

Output: -1

Explanation: It is not possible to get non-negative product in the path from (0, 0) to (2, 2), so return -1.

Example 2:

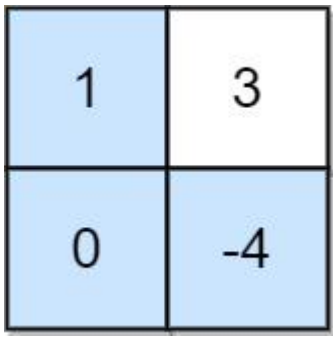


Input: grid = [[1,-2,1],[1,-2,1],[3,-4,1]]

Output: 8

Explanation: Maximum non-negative product is shown (1 * 1 * -2 * -4 * 1 = 8).

Example 3:



Input: grid = [[1,3],[0,-4]]

Output: 0

Explanation: Maximum non-negative product is shown (1 * 0 * -4 = 0).

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 15
- -4 <= grid[i][j] <= 4

```
class Solution {
public:
    int maxProductPath(vector<vector<int>>& grid) {
        const int mod = 1000000000 + 7;
        int m = grid.size(), n = grid[0].size();
        vector<vector<long long>> maxgt(m, vector<long long>(n));
        vector<vector<long long>> minlt(m, vector<long long>(n));

        maxgt[0][0] = minlt[0][0] = grid[0][0];
        for (int i = 1; i < n; i++) {
            maxgt[0][i] = minlt[0][i] = maxgt[0][i - 1] * grid[0][i];
        }
        for (int i = 1; i < m; i++) {
            maxgt[i][0] = minlt[i][0] = maxgt[i - 1][0] * grid[i][0];
        }

        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                if (grid[i][j] >= 0) {
                    maxgt[i][j] =
                        max(maxgt[i][j - 1], maxgt[i - 1][j]) * grid[i][j];
                    minlt[i][j] =
                        min(minlt[i][j - 1], minlt[i - 1][j]) * grid[i][j];
                } else {
                    maxgt[i][j] =
                        min(minlt[i][j - 1], minlt[i - 1][j]) * grid[i][j];
                    minlt[i][j] =
                        max(maxgt[i][j - 1], maxgt[i - 1][j]) * grid[i][j];
```

```
            }
        }
    }
    if (maxgt[m - 1][n - 1] < 0) {
        return -1;
    } else {
        return maxgt[m - 1][n - 1] % mod;
    }
  }
};
```